# SV9 - Digital Verification using SV & UVM

## =======Assignment (3)=======

## Functional Coverage


Name

**Ibrahim Mohamed Mohamed Hanafy**

# Question1: ALU Full Coverage

## -ALU Design Code:

```
2  module alu_seq(operand1, operand2, clk, rst, opcode, out);
3  input byte operand1, operand2;
4  input clk, rst;
5  input opcode_e opcode;
6  output byte out;
7
8  always @(posedge clk) begin
9      if (rst)
10         out <= 0;
11     else
12         case (opcode)
13             ADD: out <= operand1 + operand2;
14             SUB: out <= operand1 - operand2;
15             MULT:out <= operand1 * operand2;
16             DIV: out <= operand1 / operand2;
17             default: out <= 0;
18         endcase
19 end
20 endmodule
```

-No Bugs found

## -Verification Plan:

| Label | Design Requirement Description | Stimulus Generation | Functionality Check |
|---|---|---|---|
| ALU_1 | Reset behavior:<br>Output C must go to 0 regardless of opcode or inputs. | Directed: Assert reset at start. | Checker: if (C !== 0) error. |
| ALU_2 | All Operations<br>Additon, subtraction, multiplication, division | Pure Random | Checker: Match result per opcode cycle per operation |
| ALU_3 | Arithmetic Boundary:<br>Signed addition and subtraction<br>hitting overflow/extreme values. | Constrained Randomized:<br>Max positive (127), Max negative (-128), and Zero<br>are more often | Checker: Match result per opcode cycle. |
| ALU_4 | Multiplication operands:<br>Correct signed multiplication<br>and checking for ok overflows | Constrained Randomized:<br>Max positive (127), Max negative (-128), and Zero<br>are more often | Checker: Match result per opcode cycle. |
| ALU_5 | Division operation:<br>Considered as illegal operation | Pure Random | Checker: Match result per opcode cycle. |
| ALU_6 | Opcode Transition<br>Ensure output updates correctly when switching<br>opcodes for fixed A/B. | Fixed A and B,<br>randomly loop through opcodes. | Checker: Match result per opcode cycle. |

## - Package:

```
package testing_pkg;


    typedef enum {ADD, SUB, MULT, DIV} opcode_e;
    parameter MAX = 127;
    parameter MIN = -128;
    parameter ZERO = 0;


    class Transaction;
        rand opcode_e opcode;
        rand byte operand1;
        rand byte operand2;
        bit clk;
```

```
covergroup CovPort @(posedge clk);
    opcode_cp: coverpoint opcode
    {
        bins add_or_sub = {ADD, SUB};
        bins add_then_sub = (ADD => SUB)
        illegal_bins no_DIV = {DIV};
    }

    operand1_cp: coverpoint operand1
    {
        bins Maximum = {MAX};
        bins Minimum = {MIN};
        bins Zero = {ZERO};
        bins others = default;
    }

    large_add_sub_cp: cross operand1_cp,opcode_cp
    {
        bins max_add_sub = binsof(opcode_cp.add_or_sub)
        && binsof(operand1_cp.Maximum);
        bins min_add_sub = binsof(opcode_cp.add_or_sub)
        && binsof(operand1_cp.Minimum);
        option.weight = 5;
        option.cross_auto_bin_max = 0;
    }

endgroup
```

```
constraint Inputs {
    operand1 dist {
        MAX:=40,  //40/135
        MIN:=40,  //40/135
        ZERO:=40, //40/135
        [MIN+1:-1],[1:MAX-1]:= 15 //15/135
        };
    operand2 dist {
        MAX:=40,
        MIN:=40,
        ZERO:=40,
        [MIN+1:-1],[1:MAX-1]:= 15 //15/135
        };

}
```

# - Testbench:

```
import testing_pkg::*;
module ALU_tb;

//DUT signals
byte operand1, operand2;
reg clk, rst;
opcode_e opcode;
byte out;

//bins group instantiation
    Transaction tr = new();

//Golden model expected signals
byte expected_out;

integer error_count;
integer correct_count;

initial begin  // Create the clock (10ns period)
    clk = 0;
    forever begin
        #5 clk = ~clk;
        tr.clk = clk;
    end
end
```

```
task assert_reset_check;
    @(negedge clk)
    rst = 1;
    @(negedge clk);
    if (out !== 0) begin
        error_count = error_count + 1;
        $display("Error Reseting");
    end
    else
        correct_count = correct_count + 1;
    rst = 0;
endtask
```

```
task compute_expected;    //Golden Model
    if (opcode == ADD) expected_out = operand1 + operand2;
    else if (opcode == SUB) expected_out = operand1 - operand2;
    else if (opcode == MULT) expected_out = operand1 * operand2;
    else if (opcode == DIV) expected_out = operand1 / operand2;
    else expected_out = 8'b00;

endtask
```

```
task check_against_expected;
@(negedge clk);
if (out !== expected_out) begin
    error_count = error_count + 1;
    $display("%t: Error: For operand1=%0d, operand2=%0d, out should equal %0d but is %0d",
        $time, operand1, operand2, expected_out, out);
end
else
    correct_count = correct_count + 1;
endtask
```

```
initial begin
    error_count = 0;
    correct_count = 0;
    $display("================================================TestBench Start=======
    //1.Directed reset
    assert_reset_check;

    //2. Randomization loop
    repeat(32) begin  //take care when repeating for sequential testbenches
        assert(tr.randomize());                // as we check and stimulate after 1 tic

        operand1 = tr.operand1;
        operand2 = tr.operand2;
        opcode = opcode_e'(tr.opcode);

        @(negedge clk);
        compute_expected;
        check_against_expected;
    end

    //3. Opcode transition
    repeat(5) begin //5 times that operands fixed
        operand1 = tr.operand1;
        operand2 = tr.operand2;

        repeat(10) begin //10 transitons of opcode
            opcode = opcode_e'(tr.opcode);
            @(negedge clk);
            compute_expected;
            check_against_expected;
        end
    end

    //4.one last reset
    assert_reset_check;
    $display("%t: At end of test error count is %0d and correct count = %0d", $time, er
    $display("================================================TestBench End========
    $stop();
end
```

## - Do file

```
#vlib work
vlog -sv ../../../VS_code/Session3/Assignment3/Q1_ALU/* +cover -
covercells
vsim -voptargs=+acc work.ALU_tb -cover
#add wave *
coverage save ALU_tb.ucdb -onexit
#do wave.do
run -all

vcover report ALU_tb.ucdb -details -all -output code_cvr.txt
```

## -Code Coverage:

```
================================================================================
=== File: ../../../VS_code/Session3/Assignment3/Q1_ALU/alu.sv
================================================================================
Statement Coverage:
    Enabled Coverage            Active      Hits    Misses % Covered
    ----------------            ------      ----    ------ ---------
    Stmts                            7         6         1      85.7
```

```
***0***                                 default: out <= 0;
```

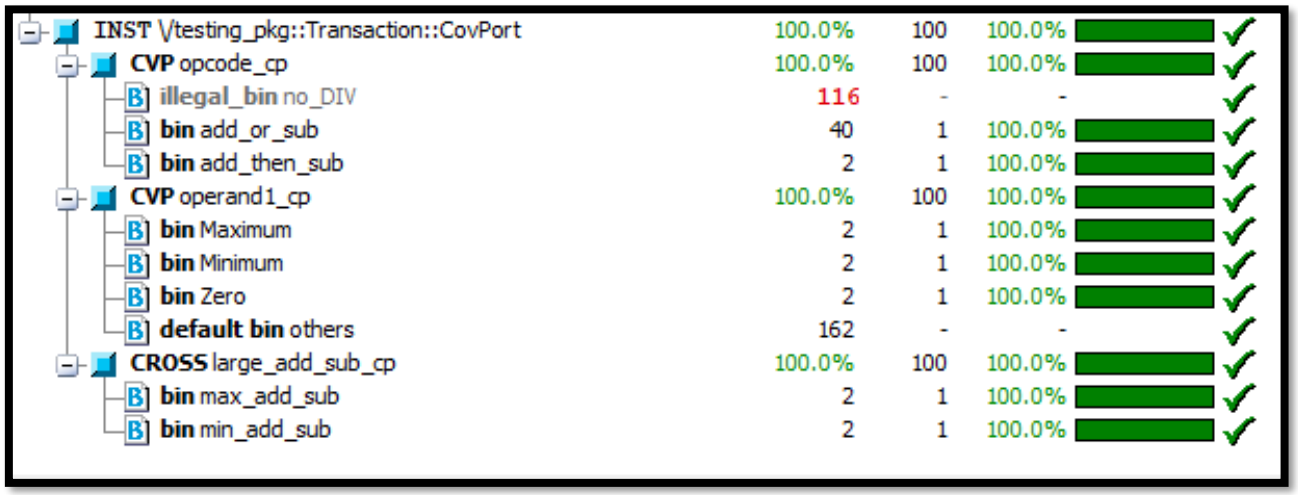
The default case of opcode not being reached should be justifiable

```
Branch Coverage:
    Enabled Coverage            Active      Hits    Misses % Covered
    ----------------            ------      ----    ------ ---------
    Branches                         7         6         1      85.7
```

```
***0***                                 default: out <= 0;
```

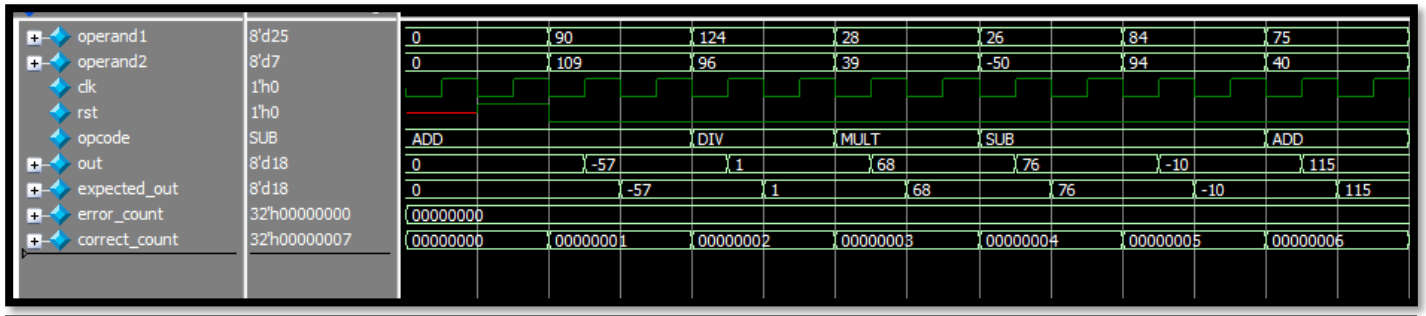
```
Toggle Coverage:
    Enabled Coverage            Active      Hits    Misses % Covered
    ----------------            ------      ----    ------ ---------
    Toggle Bins                      56        56         0     100.0
```

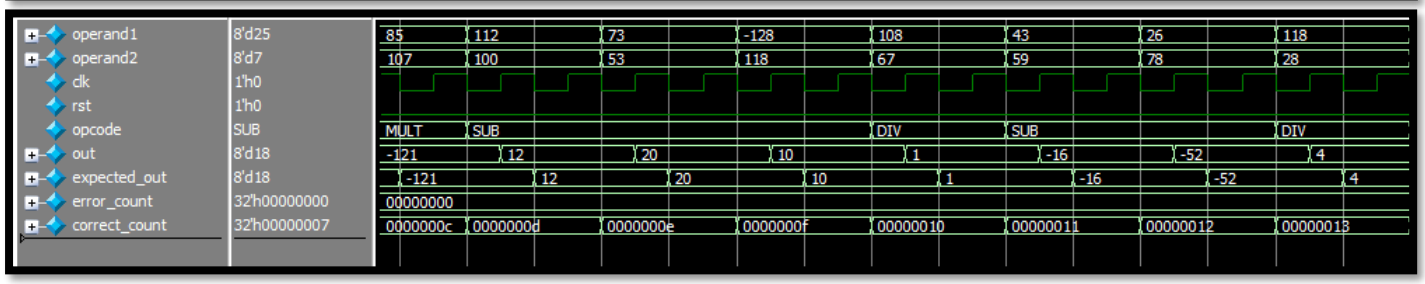
# -Code Coverage:



| | | | | |
|---|---|---|---|---|
| INST \/testing_pkg::Transaction::CovPort | 100.0% | 100 | 100.0% | ✓ |
| CVP opcode_cp | 100.0% | 100 | 100.0% | ✓ |
| illegal_bin no_DIV | 116 | - | - | ✓ |
| bin add_or_sub | 40 | 1 | 100.0% | ✓ |
| bin add_then_sub | 2 | 1 | 100.0% | ✓ |
| CVP operand1_cp | 100.0% | 100 | 100.0% | ✓ |
| bin Maximum | 2 | 1 | 100.0% | ✓ |
| bin Minimum | 2 | 1 | 100.0% | ✓ |
| bin Zero | 2 | 1 | 100.0% | ✓ |
| default bin others | 162 | - | - | ✓ |
| CROSS large_add_sub_cp | 100.0% | 100 | 100.0% | ✓ |
| bin max_add_sub | 2 | 1 | 100.0% | ✓ |
| bin min_add_sub | 2 | 1 | 100.0% | ✓ |

# -Waveforms:



# Question2: Counter

## -Counter Design Code:

```
 8  module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_
22  always @(posedge clk) begin
23      if (!rst_n)
24          count_out <= 0;
25      else if (!load_n)
26          count_out <= data_load;
27      else if (ce)
28          if (up_down)
29              count_out <= count_out + 1;
30          else
31              count_out <= count_out - 1;
32      //missing else statement infers a latch
33      //which is actually the needed implemetation
34  end
35
36  assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
37  assign zero = (count_out == 0)? 1:0;
38
39  endmodule
```

-No Bugs found

## -Verification Plan:

| Label | Design Requirement Description | Stimulus Generation | Functional Coverage | Functionality Check |
|---|---|---|---|---|
| Reset | When the reset is asserted, the output counter value should be low | Directed at the start of the simulation, then randomized such that it is off most of the time | - | A checker in the testbench to make sure the output is correct<br><br>Can also check for zero flag |
| Load | When the load is asserted, the output count_out should take the value of the load_data input | Randomization, ST:<br>It is active 70% of the time | Cover all load data values | A checker in the testbench to make sure the output is correct, matching the input |
| Enabled1 | When enabled, the counter counts up/down, count_out ouputs accordingly | Randomization, ST:<br>It is active 70% of the time | Cover all count_out values from 0 to max twice: counting up & counting down | A checker to make the ensure counting every cycle |
| Enabled2 | When enabled, the counter counts up/down, and overflows correctly | Randomization, ST:<br>It is active 70% of the time | Cover transition of overflowing twice:<br>max=>0 (up) 0=>max (down) | A checker to make the ensure counting every cycle |

## - Package:

```
class RandomControl #(parameter WIDTH = 4);;

    parameter MAX = {WIDTH{1'b1}};
    logic up_down;
    logic [WIDTH-1:0] count_out;
    logic [WIDTH-1:0] data_load;
    bit clk;

    rand logic rst_n, load_n, ce;
```

```
//constraints=========================
    constraint Reset { //off for 90% of
        rst_n dist {1:/90 , 0:/10};
        }
    constraint Load { //on for 70% of t
        load_n dist {0:/70 , 1:/30};
        }
    constraint Enable { //on for 70% of
        ce dist {1:/70 , 0:/30};
        }
```

```
covergroup CovPort @(posedge clk);
    Data_Load_cp: coverpoint data_load iff(rst_n && !load_n)
    {
        bins all_load_bins[] = {[0:$]};
    }

    counting_up_cp: coverpoint count_out iff(rst_n && ce && up_down)
    {
        bins all_count_up_bins[] = {[0:$]};
    }

    counting_up_of_cp: coverpoint count_out iff(rst_n && ce && up_down)
    {
        bins OF_max_to_zero = (MAX => 0);
    }

    counting_down_cp: coverpoint count_out iff(rst_n && ce && !up_down)
    {
        bins all_count_down_bins[] = {[0:$]};
    }

    counting_down_of_cp: coverpoint count_out iff(rst_n && ce && !up_down)
    {
        bins OF_zero_to_max = (0 => MAX);
    }
```

# - Testbench:

```
import class_pkg::*;
module Counter_tb;

//DUT Signals
parameter WIDTH = 4;
reg clk;
reg rst_n;
reg load_n;
reg up_down;
reg ce;
reg [WIDTH-1:0] data_load;

wire [WIDTH-1:0] count_out;
wire max_count;
wire zero;

//Golden model signals
reg [WIDTH-1:0] expected_count_c
reg expected_max_count;
reg expected_zero;

RandomControl #(.WIDTH(WIDTH)) cntrl = new();

counter #(.WIDTH(WIDTH)) uut (.*);
```

```
initial begin
    if (!(WIDTH inside {4,6,8})) begin
        $error("Invalid WIDTH=%0d. Allowed values are 4, 6, or 8", WIDT
        $fatal;
    end
end

integer correct_count = 0;
integer error_count = 0;

initial begin
    clk = 0;
    forever begin
        #5 clk = ~clk;
        cntrl.clk = clk;
    end
end


initial begin

    correct_count = 0; error_count = 0;
    expected_count_out = 0; expected_max_count = 0; expected_zero = 1;
```

```
task assert_reset;
    @(negedge clk)
    rst_n = 0;
    @(negedge clk);
    if (count_out !== 0) begin
        error_count = error_count + 1;
        $display("Error Reseting");
    end
    else
        correct_count = correct_count + 1;
    @(negedge clk);
    rst_n = 1;
endtask
```

```
task compute_expected; //Golden Model
    if (!rst_n)
        expected_count_out = {WIDTH{1'b0}};
    else if (!load_n)
        expected_count_out = data_load;
    else if (ce) begin
        if (up_down)
            expected_count_out = expected_count_out + 1'b1;
        else
            expected_count_out = expected_count_out - 1'b1;
    end
    else
        expected_count_out = expected_count_out; // hold value

    expected_max_count = (expected_count_out == {WIDTH{1'b1}});
    expected_zero      = (expected_count_out == {WIDTH{1'b0}});

endtask
```

```
task check_result;

    if ((count_out  !== expected_count_out) ||
        (max_count  !== expected_max_count) ||
        (zero       !== expected_zero)) begin
        error_count = error_count + 1;
        $display("==================================
        $display("Error: rst_n = %b | load_n = %b | 
        $display("Data Load = %d" , data_load);
        $display("Expected count = %d | Actual = %d",
        $display("Expected max = %b | Actual = %b  |
    end
    else
        correct_count = correct_count + 1;
endtask
```

```
//Test Series 1: Assert reset
assert_reset();

//Test Series 2: Randomization loop
repeat(1000) begin  //take care when repeating for sequential testbenches
    assert(cntrl.randomize());                // as we check and stimulate
//use obejct's values
    rst_n = cntrl.rst_n;
    load_n = cntrl.load_n;
    ce = cntrl.ce;
//fill other inputs and supply object
    up_down = $random;
    data_load = $random;
    cntrl.up_down = up_down;
    cntrl.data_load = data_load;


    compute_expected(); //Golden model

    @(negedge clk);
    cntrl.count_out = count_out;
    check_result();
    #1;

end

//Test Series 3: Assert reset again
assert_reset();
```

## - Do file

```
#vlib work
vlog -sv ../../../VS_code/Session3/Assignment3/Q2_Counter/* +cover
vsim -voptargs=+acc work.Counter_tb -cover
add wave *
coverage save Counter_tb.ucdb -onexit
#do wave.do
run -all

vcover report Counter_tb.ucdb -details -all -output code_cvr.txt
```

## -Code Coverage:

**Statement Coverage:**

| Enabled Coverage | Bins | Hits | Misses | Coverage |
|---|---|---|---|---|
| Statements | 7 | 7 | 0 | 100.00% |

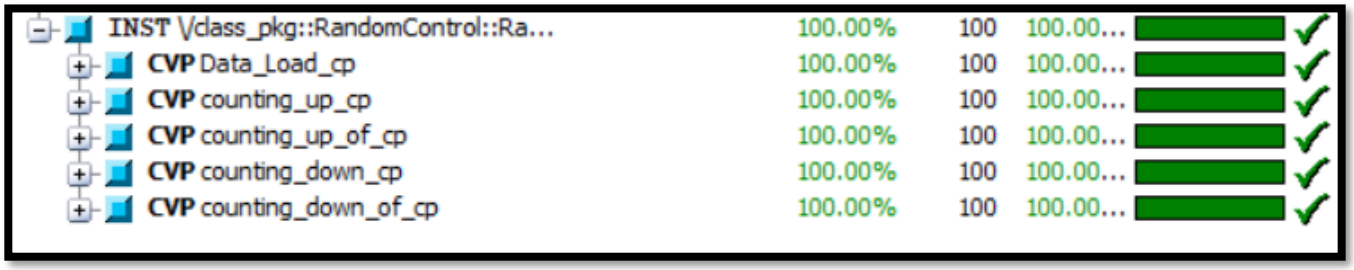
**Branch Coverage:**

| Enabled Coverage | Bins | Hits | Misses | Coverage |
|---|---|---|---|---|
| Branches | 10 | 10 | 0 | 100.00% |

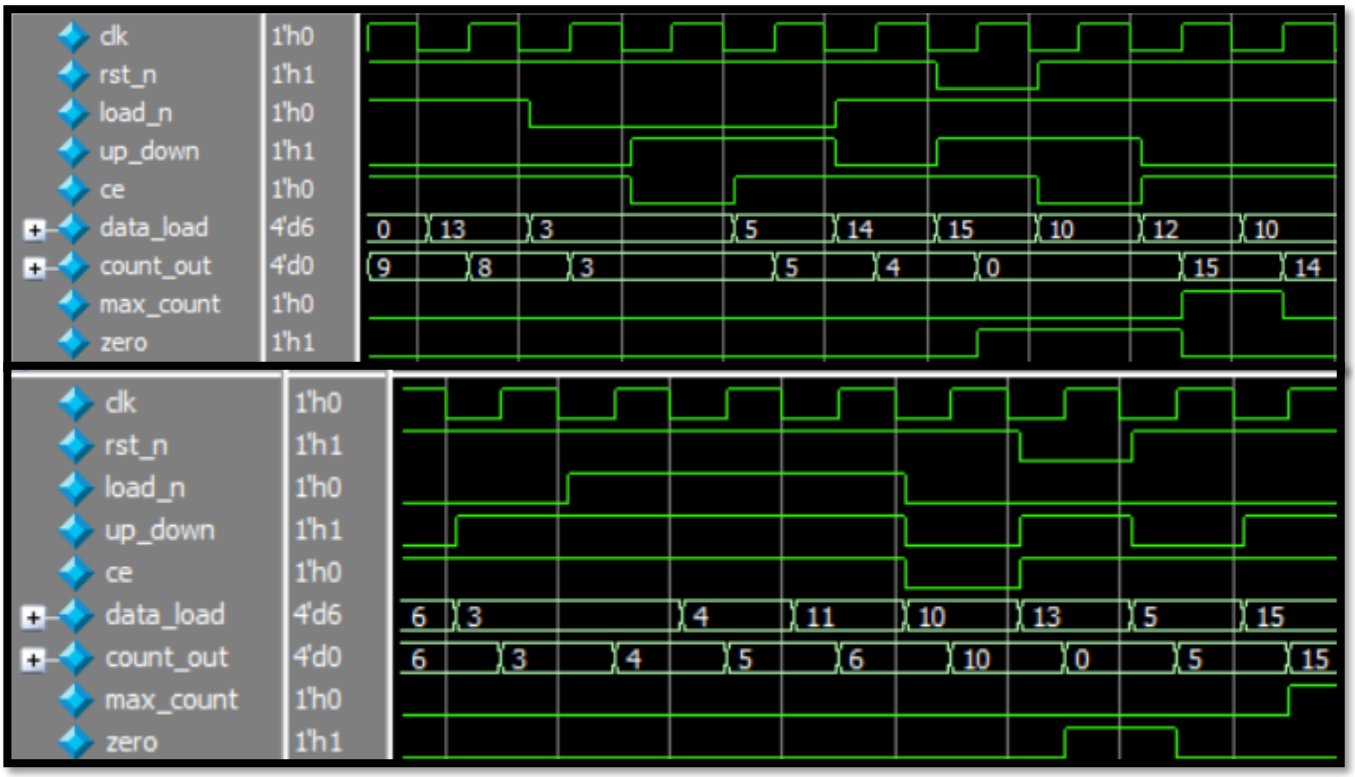
**Toggle Coverage:**

| Enabled Coverage | Bins | Hits | Misses | Coverage |
|---|---|---|---|---|
| Toggles | 30 | 30 | 0 | 100.00% |

## -Functional Coverage:




## -Waveforms:




```
==============================TestBench Start==================
 At end of test error count is 0 and correct count = 1002
==============================TestBench End====================
```

# Question3: ALSU

## -ALSU Design Code:

```
parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON"; //to allow use of cin

input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, directi
input [2:0] opcode;
input signed [2:0] A, B;

output reg [15:0] leds;
output reg signed [5:0] out;

reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, directio
reg signed [1:0] cin_reg; //cin is 1 bit<=========
reg [2:0] opcode_reg;
reg signed [2:0] A_reg, B_reg;

wire invalid_red_op, invalid_opcode, invalid;

//Invalid handling==================================================
assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[
assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
assign invalid = invalid_red_op | invalid_opcode;
```

```
always @(posedge clk or posedge rst) begin
  if(rst) begin
      cin_reg <= 0;
      red_op_B_reg <= 0;
      red_op_A_reg <= 0;
      bypass_B_reg <= 0;
      bypass_A_reg <= 0;
      direction_reg <= 0;
      serial_in_reg <= 0;
      opcode_reg <= 0;
      A_reg <= 0;
      B_reg <= 0;
  end else begin
      cin_reg <= cin;
      red_op_B_reg <= red_op_B;
      red_op_A_reg <= red_op_A;
      bypass_B_reg <= bypass_B;
      bypass_A_reg <= bypass_A;
      direction_reg <= direction;
      serial_in_reg <= serial_in;
      opcode_reg <= opcode;
      A_reg <= A;
      B_reg <= B;
  end
end
```

```
//leds output blinking=====================
always @(posedge clk or posedge rst) begin
  if(rst) begin
      leds <= 0;
  end else begin
      if (invalid)
        leds <= ~leds;
      else
        leds <= 0;
  end
end
```

```
always @(posedge clk or posedge rst) begin
  if(rst) begin
    out <= 0;
  end
  else begin
    if (bypass_A_reg && bypass_B_reg)
      out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
    else if (bypass_A_reg)
      out <= A_reg;
    else if (bypass_B_reg)
      out <= B_reg;
    else if (invalid)
        out <= 0;
    else begin
        case (opcode) //Should use the registered opcode here<=====
          3'h0: begin //OR
            if (red_op_A_reg && red_op_B_reg)
              out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
            else if (red_op_A_reg)
              out <= |A_reg;
            else if (red_op_B_reg)
              out <= |B_reg;
            else
              out <= A_reg | B_reg;
          end
          3'h1: begin //XOR
            if (red_op_A_reg && red_op_B_reg)
              out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
            else if (red_op_A_reg)
              out <= ^A_reg;
            else if (red_op_B_reg)
              out <= ^B_reg;
            else
              out <= A_reg ^ B_reg;
          end
          3'h2: out <= A_reg + B_reg; // cin wasnt taken into considerat
          3'h3: out <= A_reg * B_reg;
          3'h4: begin //SHIFT
            if (direction_reg)
              out <= {out[4:0], serial_in_reg};
            else
              out <= {serial_in_reg, out[5:1]};
          end
          3'h5: begin //ROTATE
            if (direction_reg)
              out <= {out[4:0], out[5]};
            else
              out <= {out[0], out[5:1]};
          end //3'h6 3'h7 should have been included<====
        endcase
  end
```

# -Bugs & Fixes

-Adder didn’t implement cin where we check if FULL_ADDER is ON

-Registered opcode wasn’t used in case block

-cin_reg size is wrong

-cases 3’h6 & 3’h7 weren’t stated

# -Verification Plan:

| Label | Design Requirement Description | Stimulus Generation | Functional Coverage | Functionality Check |
|---|---|---|---|---|
| Reset | When the reset is asserted, the output counter value should be low | Directed at the start of the simulation,<br>Then randomized in a loop and constrainted to be off most of the time | - | A checker in the testbench to make sure the output is zero |
| Addition | If not Invalid:<br>Additon should be correct with availability of carry cin if FULL_ADDER is ON | Randomization while constrained inputs :<br>MAXPOS \| MAXNEG \| ZERO most of the time | Have max, min and zero corner bins to cover as well as a default bin | verified against the golden model |
| OR / XOR | When their opcode is set:<br>If Reduction set: According to priority<br>preform reduction operation | Randomized in a loop and constrainted such that for reduction:<br>the selected i/p has one bit high and the unselected one to have all low bits | **Have all walking one values as bins to cover**<br>-Where the A bins are active **if** red_op_A set<br>-And the A bins are active if red_op_B set and red_op_A low | verified against the golden model<br>reduction should now always yield 1 |
| OR / XOR | When their opcode is set:<br>If NO reduction set:<br>preform bitwise operation in i/p(s) | Randomized in a loop, no constraints | - | verified against the golden model<br>reduction should now always yield 1 |
| Addition / Multipication | If not Invalid:<br>Additon should be correct with availability of carry cin if FULL_ADDER is ON | Randomization while constrained inputs :<br>MAXPOS \| MAXNEG \| ZERO most of the time | Have max, min and zero corner bins to cover as well as a default bin | verified against the golden model |
| Validity | -Opcode bits are set to 110 or 111<br>-red_op_A or red_op_B are high and the opcode is not OR or XOR operation<br>Both yield invalid | Constrained such that invalid/ illegal cases are less frequent than valid ones | - | LED blinking functionality |
| bypass | The i/p is simply passed as o/p | Constrained such that bypasses aren't frequent | - | Check o/p vs i/p |
| Shift | Shift the output according to the direction input by 1 bit | Pure Random: No Constraints | - | verified against the golden model |
| Rotate | Rotate the output according to the direction input by 1 bit | Pure Random: No Constraints | - | verified against the golden model |
| Opcodes | according to each opcode preform the assigned function<br>opcodes 6,7 aren't assigned to anything and considered invalid | Constrainted ST: invalid opcodes are genereated less often | Bins_shift[] \|\| Bins_arith[] \|\| Bins_bitwise[]<br>illegal bins for invalid opcodes<br>Transition bin of opcode form 0>1>....>5 | Preform each function specified by the spec |

# - Package:

```
package class_pkg;

    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";

    typedef enum reg [2:0] {OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID_6, INVALID_7} opcode_e;
    localparam MAXPOS = 3, ZERO = 0, MAXNEG = -4;


    class RandomControl;
        rand logic rst,red_op_A, red_op_B, bypass_A, bypass_B, cin, direction, serial_in;
        rand logic [2:0] opcode;
        rand logic signed [2:0] A, B;

        bit clk;
        rand opcode_e my_opcodes[6];

```

```
//=====================================CONSTRAINTS=====================
    constraint Reset { //off for 90% of the time
        rst dist {0:/90 , 1:/10};
    }

    constraint Input { //MAXs and Zero 80% of the time for ADD and MULT
        if((opcode == ADD) | (opcode == MULT)){
            A dist {MAXPOS:=40,
                MAXNEG:=40,
                ZERO:=40,
                [-3:-1], [1:2] := 15};
            B dist {MAXPOS:=40,
                MAXNEG:=40,
                ZERO:=40,
                [-3:-1], [1:2] := 15}; //15/135
        }
    }

    constraint Bypass { //off for 90% of the time
        bypass_A dist {0:/90 , 1:/10};
        bypass_B dist {0:/90 , 1:/10};
    }

    constraint Valdity { //Invalid for 10% of the time
        opcode dist {INVALID_6:/10,INVALID_7:/10,
                     [OR:ROTATE]:/80};
        if(opcode ==  (OR | XOR)){
            red_op_A dist{0:/50 , 1:/50};  //both red high is unlikely?
            if(red_op_A){
                red_op_B dist{0:/80 , 1:/20};
            }
        }
    }

    constraint Reduction_Operations {       //selected has one bit set (
        if(opcode inside {OR, XOR}){  //unslected has all low
            if(red_op_A){
                A dist { 4:=80, 1:=80, 2:=80, [-3:0]:=20, 3:=20 };
                B == 3'b000;
            }
            else if(red_op_B){
                B dist { 4:=80, 1:=80, 2:=80, [-3:0]:=20, 3:=20 };
                A == 3'b000;
            }
        }
```

```
    constraint opcode_rand_array {//"constraint 8"
        unique {my_opcodes};
        foreach(my_opcodes[i]){
            my_opcodes[i] inside {[OR:ROTATE]};
        }

    }
```

```
//=========================================COVER GROUPS=================================
covergroup CovPort; //@(posedge clk)
    A_cp: coverpoint A iff(1)
    {
        bins A_data_0 = {ZERO};
        bins A_data_max = {MAXPOS};
        bins A_data_min = {MAXNEG};
        bins A_data_default = default;
        bins A_data_walkingones[] = {3'b001,3'b010,3'b100} iff(red_op_A);
    }

    B_cp: coverpoint B iff(1)
    {
        bins B_data_0 = {ZERO};
        bins B_data_max = {MAXPOS};
        bins B_data_min = {MAXNEG};
        bins B_data_default = default;
        bins B_data_walkingones[] = {3'b001,3'b010,3'b100} iff(!red_op_A && red_op_B);
    }

    ALU_cp: coverpoint opcode iff(1)
    {
        bins Bins_shift[] = {SHIFT,ROTATE};
        bins Bins_arith[] = {ADD, MULT};
        bins Bins_bitwise[] = {OR, XOR};
        illegal_bins Bins_invalid = {INVALID_6, INVALID_7};
        bins Bins_trans = (0 => 0 => 1 => 1 => 2 => 2 => 3 => 3 => 4 => 4 => 5 => 5);
        //the transition sequence is like this as we wait two cycles before changing
        //opcode in the testbench
    }
```

```
function new();
    CovPort = new;
endfunction


endclass
```

## - Testbench:

```
reg signed [5:0] expected_out;
reg [15:0] expected_leds;


ALSU #(
    .INPUT_PRIORITY(INPUT_PRIORITY),
    .FULL_ADDER(FULL_ADDER)
    ) uut (.*);
ALSU_golden #(
    .INPUT_PRIORITY(INPUT_PRIORITY),
    .FULL_ADDER(FULL_ADDER)
    ) golden (.out(expected_out),.leds(expected_leds),.*
```

```
//Class and Objects
RandomControl cntrl = new();

//clk==========================
initial begin
    clk = 0;
    forever begin
        #5 clk = ~clk;
        cntrl.clk = clk;
    end
end
```

```
//Sampling for Bins==========================
always @(posedge clk)
    cntrl.CovPort.sample();
always @(posedge rst or posedge bypass_A or posedge bypass_B)
    cntrl.CovPort.stop();
always @(negedge rst or negedge bypass_A or negedge bypass_B) begin
    if(!rst && !bypass_A && !bypass_B)
        cntrl.CovPort.start();
    else
        cntrl.CovPort.stop();
end
```

```
task assert_reset;
    @(negedge clk);
    rst = 1;
    @(negedge clk);
    @(negedge clk);
    if (expected_out !== 0) begin
        error_count = error_count + 1;
        $display("Error Reseting");
    end
    else
        correct_count = correct_count + 1;
    @(negedge clk);
    rst = 0;
endtask
```

```
task check_result;

    if((out  == expected_out) && (leds == expected_leds))
    correct_count = correct_count + 1;
    else
    error_count = error_count + 1;

    if (out  !== expected_out) begin
        $display("================================================================================");
        $display("Error: opcode = %s | bypass_A = %b | bypass_B = %b | red_op_A = %b | red_op_B = %b
        $display("direction = %b | serial_in = %b" , direction,serial_in);
        $display("A: %d | B: %d", A, B);
        $display("Expected out = %d | Actual = %d", out, expected_out);
    end

    if (leds  !== expected_leds) begin
        $display("================================================================================");
        $display("Error: opcode = %s | bypass_A = %b | bypass_B = %b | red_op_A = %b | red_op_B = %b
        $display("direction = %b | serial_in = %b" , direction,serial_in);
        $display("A: %d | B: %d", A, B);
        $display("Expected leds = %d | Actual = %d", leds, expected_leds);
    end

endtask
```

```
initial begin

    correct_count = 0; error_count = 0;
    expected_out = 0; expected_leds = 0;
    $display("=================================================


    //Test Series 1: Assert reset
    assert_reset();

    //Test Series 2: [First loop: constraint 8 is off]
    cntrl.opcode_rand_array.constraint_mode(0);
    repeat(1000) begin  //take care when repeating for sequential
        assert(cntrl.randomize());                // as we check
    //use obejct's values
        A = cntrl.A;
        B = cntrl.B;
        opcode = opcode_e'(cntrl.opcode);

        cin = cntrl.cin; rst = cntrl.rst ;
        red_op_A = cntrl.red_op_A; red_op_B = cntrl.red_op_B;
        bypass_A = cntrl.bypass_A; bypass_B = cntrl.bypass_B;
        direction= cntrl.direction; serial_in = cntrl.serial_in;

    //wait for two clock cycles
        @(negedge clk);
        @(negedge clk);

        check_result();
        #1;

    end
```

```
//Test Series 3: [Second loop: constraint 8 only on, else di
//no bypasses,reduction or resetting
cntrl.constraint_mode(0);
cntrl.opcode_rand_array.constraint_mode(1);
repeat(3000) begin  //take care when repeating for sequentia
    assert(cntrl.randomize());                // as we check
//use obejct's values
    A = cntrl.A;
    B = cntrl.B;

    cin = cntrl.cin; rst = 0;
    red_op_A = 0; red_op_B = 0;
    bypass_A = 0; bypass_B = 0;
    direction= cntrl.direction; serial_in = cntrl.serial_in;

    foreach(cntrl.my_opcodes[i])begin //6 iterations * 1000
        cntrl.opcode = cntrl.my_opcodes[i]; //all valid from
        opcode = opcode_e'(cntrl.opcode); //we didnt drive o
        //wait for two clock cycles
        @(negedge clk); //all 6 iterations share the same i/
        @(negedge clk);

        check_result();
        #1;
    end
end
```

## - Do file

```
#vlib work
vlog -sv ../../../VS_code/Session3/Assignment3/Q3_ALSU/* +cover -covercells
vsim -voptargs=+acc work.ALSU_tb -cover
#add wave *
coverage save ALSU_tb.ucdb -onexit
do wave.do
run -all

vcover report ALSU_tb.ucdb -details -all -output code_cvr.txt
```

## -Code Coverage:

```
================================================================================
=== File: ../../../VS_code/Session3/Assignment3/Q3_ALSU/ALSU.v
================================================================================
Statement Coverage:
    Enabled Coverage              Active      Hits    Misses % Covered
    ----------------              ------      ----    ------ ---------
    Stmts                             51        49         2      96.0
```

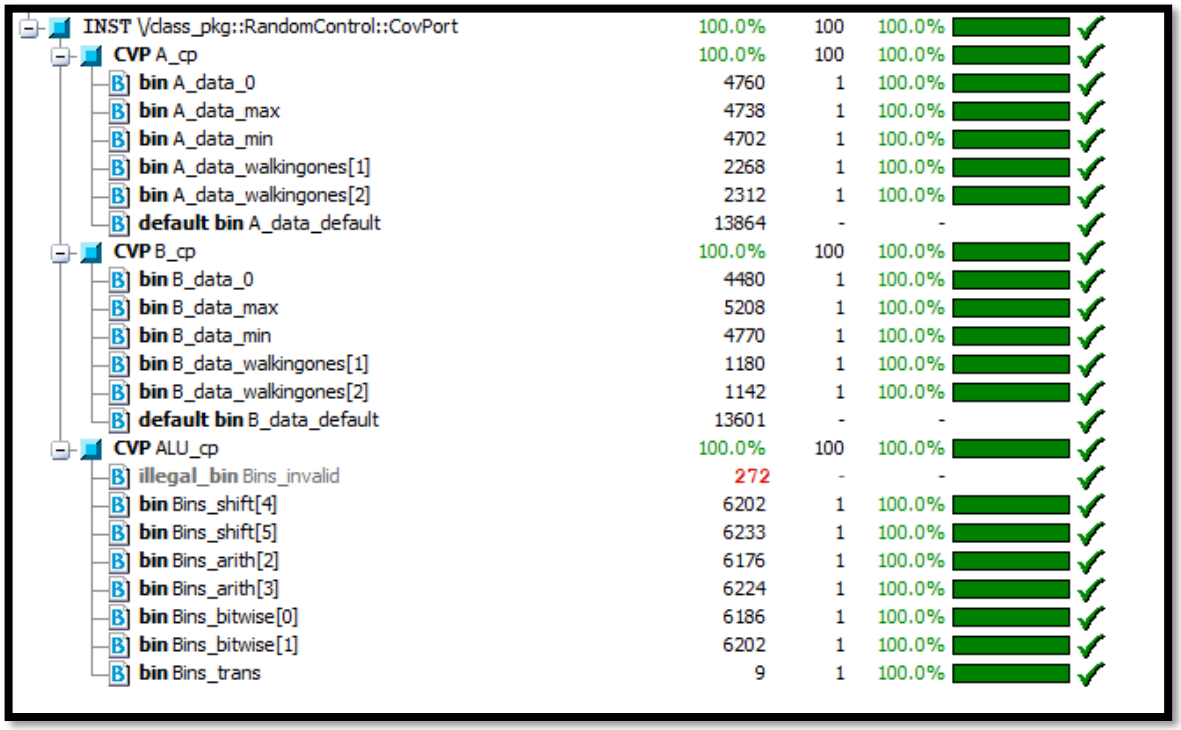
```
Branch Coverage:
    Enabled Coverage              Active      Hits    Misses % Covered
    ----------------              ------      ----    ------ ---------
    Branches                          34        32         2      94.1
```

```
***0***                 3'h6: out <= 0;
***0***                 3'h7: out <= 0;
```

-The two invalid opcode (statements/Branches) as the design's code structure checks for invalidity before these statements

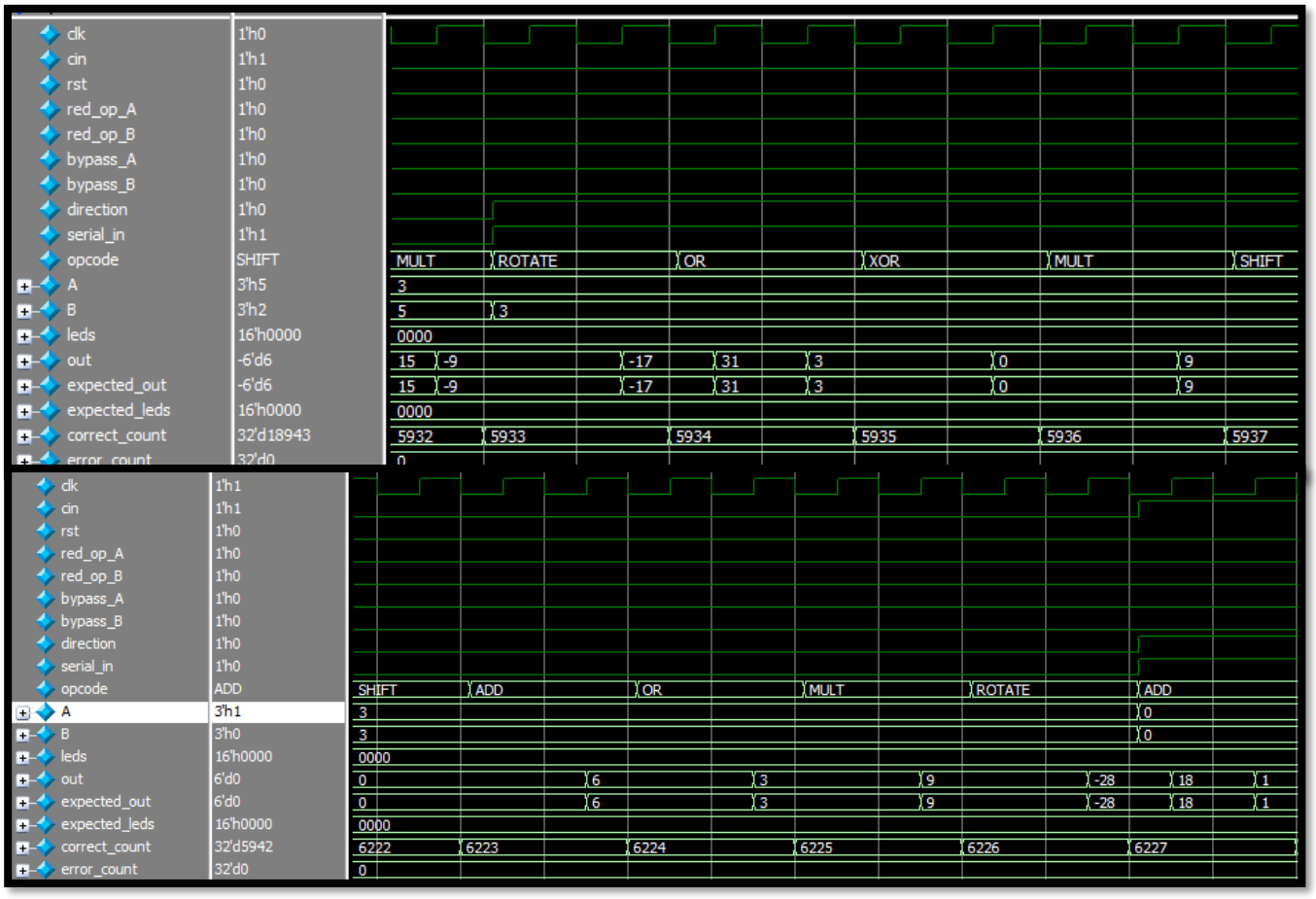
```
Toggle Coverage:
    Enabled Coverage              Active      Hits    Misses % Covered
    ----------------              ------      ----    ------ ---------
    Toggle Bins                      118       118         0     100.0
```

# -Functional Coverage:



| | | | |
|---|---|---|---|
| INST \/class_pkg::RandomControl::CovPort | 100.0% | 100 | 100.0% |
| CVP A_cp | 100.0% | 100 | 100.0% |
| bin A_data_0 | 4760 | 1 | 100.0% |
| bin A_data_max | 4738 | 1 | 100.0% |
| bin A_data_min | 4702 | 1 | 100.0% |
| bin A_data_walkingones[1] | 2268 | 1 | 100.0% |
| bin A_data_walkingones[2] | 2312 | 1 | 100.0% |
| default bin A_data_default | 13864 | - | - |
| CVP B_cp | 100.0% | 100 | 100.0% |
| bin B_data_0 | 4480 | 1 | 100.0% |
| bin B_data_max | 5208 | 1 | 100.0% |
| bin B_data_min | 4770 | 1 | 100.0% |
| bin B_data_walkingones[1] | 1180 | 1 | 100.0% |
| bin B_data_walkingones[2] | 1142 | 1 | 100.0% |
| default bin B_data_default | 13601 | - | - |
| CVP ALU_cp | 100.0% | 100 | 100.0% |
| illegal_bin Bins_invalid | 272 | - | - |
| bin Bins_shift[4] | 6202 | 1 | 100.0% |
| bin Bins_shift[5] | 6233 | 1 | 100.0% |
| bin Bins_arith[2] | 6176 | 1 | 100.0% |
| bin Bins_arith[3] | 6224 | 1 | 100.0% |
| bin Bins_bitwise[0] | 6186 | 1 | 100.0% |
| bin Bins_bitwise[1] | 6202 | 1 | 100.0% |
| bin Bins_trans | 9 | 1 | 100.0% |

# -Waveforms:



# Question4: RAM

## -RAM Design Code:

```
module my_mem(
input clk,
input write,
input read,
input [7:0] data_in,
input [15:0] address,
output reg [7:0] data_out
);
// Declare a 9-bit associative array using the logic data type

logic [8:0] mem_array [int];

always @(posedge clk) begin
if (write)
mem_array[address] = {~^data_in, data_in}; //write 9 bits
else if (read)
data_out =  mem_array[address];  //read 8 bits
end
endmodule
```

## -Verification Plan:

| Label | Design Requirement Description | Stimulus Generation | Functionality Check |
|---|---|---|---|
| Writing | When write is high, data in is saved as the data in the address given at the positive edge of clk | Directed when writing | When writing, also write same data to a golden model, and later read the data that was written |
| Reading | When read is high, dataout outputs the data in the address given at the positive edge of clk | Directed when reading | Verified against a golden model that was written to as well |
| data_in | When write is high, data in is saved as the data in the address given at the positive edge of clk | A dynamic array contains all data to be wrritten, where its elements are randomized | Verified against a golden model that was written to as well |
| address | "" "" "" "" "" "" | A dynamic array contains all addresses to be wrritten to, where its elements are randomized | Verified against a golden model that was written to as well |

## - Package:

```
package class_pkg;

    class RandomControl;
        logic [15:0] addresses_array[];  //dynamic to store random addresses
        logic [7:0] data_to_write_array[]; //corresponding data to write
        rand logic [15:0] random_address;
        rand logic [7:0] random_data_to_write;


        logic [7:0] data_read_expect_assoc[int]; //expected read (8 bits)
        logic [7:0] data_read_queue[$];  //actual read (8 bits)

        bit clk;
```

## - Testbench:

```
import class_pkg::*;
module RAM_tb;

localparam TESTS = 100;


reg clk, write, read;
reg [7:0] data_in;
reg [15:0] address;
wire [7:0] data_out;

my_mem uut (.*);

integer correct_count = 0;
integer error_count = 0;
```

```
//Class and Objects
RandomControl cntrl = new();

//clk=========================
initial begin
    clk = 0;
    forever begin
        #5 clk = ~clk;
        cntrl.clk = clk;
    end
end

integer i;
```

```
task stimulus_gen;

    cntrl.addresses_array = new[TESTS];
    cntrl.data_to_write_array = new[TESTS];
    //integer i;
    for(i=0 ; i < TESTS ; i++) begin
        assert(cntrl.randomize());
        cntrl.addresses_array[i] = cntrl.random_address;
        cntrl.data_to_write_array[i] = cntrl.random_data_to_write;
    end

endtask
```

```
task golden_model;

    //integer i;
    for(i=0 ; i < TESTS ; i++) begin  //loop to populate the expected read
        cntrl.data_read_expect_assoc[cntrl.addresses_array[i]] =
        cntrl.data_to_write_array[i];
        //copying the data write exactly is golden behaviour
    end

endtask
```

```
task check9Bits ();

    if(data_out  === cntrl.data_read_expect_assoc[address])
    correct_count = correct_count + 1;
    else
    error_count = error_count + 1;

    if (data_out  !== cntrl.data_read_expect_assoc[address]) begin
        $display("==========================================================
        $display("Error: write = %b | read = %b" , write, read);
        $display("address = %b |  data_in = %b" , address , data_in);
        $display("Expected out = %d | Actual = %d", cntrl.data_read_expect_assoc[ad
    end

endtask
```

```
initial begin

    $display("=================================================TestBench Start=====

    stimulus_gen(); //stimulii are generated
    golden_model(); //golden model expected calculated

    write = 1; read = 0;    //writing data on DUT

    for(i = 0 ; i < TESTS ; i++) begin
        @(negedge clk);
        address = cntrl.addresses_array[i];
        data_in = cntrl.data_to_write_array[i];

    end
    @(negedge clk); //wait for last write


    write = 0; read = 1;              //reading from DUT

    for(i = 0 ; i < TESTS ; i++) begin
        address = cntrl.addresses_array[i];
        @(negedge clk);
        check9Bits();
        cntrl.data_read_queue[i] = data_out; //saving DUT memory
    end

    $display("--------- Final Data Read Queue Contents --------"); //Completion and R

    //check if elements remain in the queue
    while (cntrl.data_read_queue.size() > 0) begin
        logic [7:0] temp_data;
        temp_data = cntrl.data_read_queue.pop_front();

        $display("Popped Data: %h", temp_data);
    end


    $display("%t: At end of test error count is %0d and correct count = %0d", $time,
    $display("=================================================TestBench End======
    $stop;

end
```

## - Do file

```
#vlib work
vlog -sv ../../../VS_code/Session3/Assignment3/Q4_RAM/* +cover -
covercells
vsim -voptargs=+acc work.RAM_tb -cover
#add wave *
coverage save RAM_tb.ucdb -onexit
do wave.do
run -all

vcover report RAM_tb.ucdb -details -all -output code_cvr.txt
```

## -Waveforms: